

Jobsheet 7 — PHP Dasar & Form Handling

Sub-CPMK: Mengimplementasikan dasar PHP & pengolahan form.

Perubahan dari Jobsheet 6

- Semua halaman `.html` diubah menjadi `.php`.
- Diperkenalkan `includes/header.php` & `includes/footer.php` untuk menghindari duplikasi navbar/footer di setiap halaman (dipakai lewat `include`).
- Path CSS/JS/menu memakai path **relatif** (`assets/css/style.css`, `index.php`, `dst`, tanpa awalan `/`), dihitung otomatis di `includes/header.php` berdasarkan kedalaman folder halaman yang sedang diakses (`$base = ""` di root, `"../"` untuk halaman satu level ke dalam seperti `buku/`, `anggota/`). Jadi proyek ini tetap berjalan benar walau diakses dari root server (`php -S`) **maupun** lewat subfolder (mis. Laragon dengan document root di folder induk).
- `buku/tambah.php` & `anggota/tambah.php`: form kini `method="post"` mengarah ke `proses_tambah.php` masing-masing.
- `buku/proses_tambah.php` & `anggota/proses_tambah.php`: memvalidasi `$_POST` di server (validasi ini **terpisah** dari validasi JS di Jobsheet 5 — bisa berjalan sendiri walau JS dimatikan), lalu menyimpan sementara ke `$_SESSION['buku']` / `$_SESSION['anggota']` (array), redirect ke `list.php`.
- `buku/list.php` & `anggota/list.php`: tabel dirender dari `$_SESSION` via `foreach` (menggantikan pendekatan fetch/JSON di Jobsheet 6 — rendering utama sekarang di server).
- Flash message sukses/gagal ditampilkan lewat `$_SESSION['flash']`.
- File `assets/js/buku.js`, `assets/js/anggota.js`, dan folder `data/` dari Jobsheet 6 **dihapus** karena rendering sudah dipindah ke server-side PHP.

Cara menjalankan

Opsi 1 — PHP built-in server, jalankan dari dalam folder `jobsheet-07/`:

```
php -S localhost:8000
```

Buka `http://localhost:8000/index.php`.

Opsi 2 — Laragon (Apache): bisa lewat virtual host yang document root-nya langsung ke folder `jobsheet-07/` (mis. `http://jobsheet07.test/`), atau diakses bersarang di bawah domain proyek

(mis. <http://dp2026.test/kode-praktikum/jobsheet-07/>) — dua-duanya jalan karena path CSS/JS/link sudah relatif otomatis (lihat catatan di atas).

Catatan

- Data yang disimpan di `$_SESSION` akan hilang saat sesi browser berakhir — ini jembatan sementara. Mulai Jobsheet 8, penyimpanan dipindah ke PostgreSQL agar persisten.
- Coba nonaktifkan JavaScript di browser lalu submit form kosong: validasi server tetap mencegah data invalid tersimpan.

Wireframe & User Flow — SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota — Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow — Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow — Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]
-> [Kembali ke Dashboard]
```

Wireframe: Halaman Login

SIMPUS-Mini

[Login Petugas]

Username : [_____]

Password : [_____]

[Masuk]

Belum punya akun? Daftar di sini

Wireframe: Dashboard Petugas

SIMPUS-Mini Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |

[Total Buku] [Total Anggota] [Sedang Dipinjam]

Aksi Cepat:

[+ Peminjaman Baru] [+ Pengembalian]

Transaksi Terbaru

Anggota | Buku | Tgl Pinjam | Status

Wireframe: Form Peminjaman

Form Peminjaman Buku

Anggota : [dropdown pilih anggota]

```

| Buku      : [ dropdown, hanya stok>0 ]|
| Tanggal Pinjam : [ auto: hari ini ] |
|
|          [ Simpan Peminjaman ]      |
+-----+

```

Wireframe: Form Pengembalian

```

+-----+
| Pengembalian Buku |
+-----+
| Cari transaksi aktif: |
| [ nama anggota / judul buku _____ ]|
|
| Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+

```

Wireframe: Riwayat Peminjaman per Anggota

```

+-----+
| Riwayat Peminjaman - Siti Aminah |
+-----+
| Buku      | Pinjam   | Kembali | Status |
| Laskar Pelangi | 01/07   | 10/07   | Selesai |
| Bumi Manusia  | 15/07   | -       | Dipinjam |
+-----+

```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Dokumentasi Jobsheet 7 – PHP Dasar & Form Handling

Dokumentasi ini melanjutkan [dokumentasi jobsheet-06](#) (Fetch API & JSON). Jobsheet-07 adalah **titik paling besar** dalam perjalanan SIMPUS-Mini sejauh ini: aplikasi berpindah dari yang sepenuhnya berjalan di **browser** (HTML/CSS/JS statis) menjadi aplikasi yang punya **server** **sebenarnya** di baliknya, memakai **PHP**.

Tentang docs/wireframe.md

File ini **identik persis** dengan [docs/wireframe.md](#) [di jobsheet-06](#) — tidak ada rancangan UI/UX baru di jobsheet ini. Baca [dokumentasi jobsheet-04](#) kalau perlu menyegarkan ingatan soal wireframe & user flow.

Kenapa Ini Perubahan Besar?

Semua jobsheet sebelumnya (01-06) bisa dijalankan hanya dengan **membuka file di browser** — bahkan jobsheet-06 yang butuh server lokal pun sebenarnya server itu hanya untuk melayani file statis (HTML, CSS, JS, JSON) apa adanya, tanpa mengolah apa pun. Di jobsheet-07, untuk **pertama kalinya**, ada kode yang benar-benar **dijalankan di server** sebelum halaman dikirim ke browser: PHP memproses data, mengambil keputusan (misalnya "apakah form ini valid?"), dan **menghasilkan** HTML yang berbeda-beda tergantung situasinya (misalnya menampilkan pesan error atau tidak) — bukan sekadar mengirim file HTML yang sudah jadi apa adanya seperti jobsheet-jobsheet sebelumnya.

Apa yang Baru di Jobsheet 7?

Sesuai [README.md](#) jobsheet ini:

1. Semua halaman `.html` diubah jadi `.php`.
2. Dua file baru, `includes/header.php` dan `includes/footer.php`, menghindari duplikasi navbar/footer lewat `include`.
3. Path CSS/JS/menu di `includes/header.php` / `footer.php` dihitung **relatif otomatis** lewat variabel `$base` (lihat [bab 2 §2.3](#)), supaya tetap benar dipakai bersama oleh halaman di kedalaman folder yang berbeda-beda.
4. Form Tambah Buku/Anggota kini benar-benar **mengirim data** ke `proses_tambah.php` (bukan lagi form kosong tanpa `action` seperti sejak jobsheet-01).

5. `proses_tambah.php` memvalidasi data di **server**, menyimpannya ke `$_SESSION`, lalu redirect ke halaman daftar.
6. `list.php` merender tabel dari `$_SESSION`, menggantikan pendekatan `fetch/JSON` di jobsheet-06.
7. **Flash message** — pesan sukses/gagal yang muncul sekali setelah redirect.
8. `assets/js/buku.js`, `assets/js/anggota.js`, dan folder `data/` dari jobsheet-06 **dihapus** — tidak dibutuhkan lagi karena rendering sudah pindah ke server.

Daftar Isi

1. [Konsep Dasar PHP](#)
2. [includes/header.php & includes/footer.php](#)
3. [Session & Alur Data](#)
4. [Memproses Form: `proses\_tambah.php`](#)
5. [Menampilkan Data: `list.php` & Flash Message](#)
6. [CSS: Gaya Flash Message](#)
7. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-07/
├── index.php                # Beranda, kini file PHP
├── includes/
│   ├── header.php          # BARU — bagian atas HTML + navbar, dipakai ulang
│   └── footer.php          # BARU — bagian bawah HTML + footer, dipakai ulang
├── assets/
│   ├── css/style.css       # Ditambah gaya .flash
│   └── js/app.js           # Tidak berubah dari jobsheet-06
├── buku/
│   ├── list.php            # Render dari $_SESSION, bukan lagi fetch/JSON
│   ├── tambah.php         # Form kini punya method="post" & action
│   └── proses_tambah.php   # BARU — validasi server + simpan ke $_SESSION
├── anggota/
│   ├── list.php
│   ├── tambah.php
│   └── proses_tambah.php   # BARU
└── docs/wireframe.md       # Identik dengan jobsheet-06
```

Catatan penting dari [README.md](#) jobsheet ini yang perlu diingat sejak awal: data yang disimpan di `$_SESSION` **akan** hilang** begitu sesi browser berakhir (menutup browser, atau sesi** kedaluwarsa) — ini jembatan **sementara** menuju penyimpanan sungguhan. Mulai Jobsheet 8, data akan dipindah ke database PostgreSQL supaya benar-benar tersimpan permanen.

Dokumentasi Jobsheet 7 — PHP Dasar & Form Handling

Dokumentasi ini melanjutkan [dokumentasi jobsheet-06](#) (Fetch API & JSON). Jobsheet-07 adalah **titik paling besar** dalam perjalanan SIMPUS-Mini sejauh ini: aplikasi berpindah dari yang sepenuhnya berjalan di **browser** (HTML/CSS/JS statis) menjadi aplikasi yang punya **server sungguhan** di baliknya, memakai **PHP**.

Tentang docs/wireframe.md

File ini **identik persis** dengan [docs/wireframe.md](#) **di jobsheet-06** — tidak ada rancangan UI/UX baru di jobsheet ini. Baca [dokumentasi jobsheet-04](#) kalau perlu menyegarkan ingatan soal wireframe & user flow.

Kenapa Ini Perubahan Besar?

Semua jobsheet sebelumnya (01-06) bisa dijalankan hanya dengan **membuka file di browser** — bahkan jobsheet-06 yang butuh server lokal pun sebenarnya server itu hanya untuk melayani file statis (HTML, CSS, JS, JSON) apa adanya, tanpa mengolah apa pun. Di jobsheet-07, untuk **pertama kalinya**, ada kode yang benar-benar **dijalankan di server** sebelum halaman dikirim ke browser: PHP memproses data, mengambil keputusan (misalnya "apakah form ini valid?"), dan **menghasilkan** HTML yang berbeda-beda tergantung situasinya (misalnya menampilkan pesan error atau tidak) — bukan sekadar mengirim file HTML yang sudah jadi apa adanya seperti jobsheet-jobsheet sebelumnya.

Apa yang Baru di Jobsheet 7?

Sesuai [README.md](#) jobsheet ini:

1. Semua halaman `.html` diubah jadi `.php`.
2. Dua file baru, `includes/header.php` dan `includes/footer.php`, menghindari duplikasi navbar/footer lewat `include`.
3. Path CSS/JS/menu di `includes/header.php` / `footer.php` dihitung **relatif otomatis** lewat variabel `$base` (lihat [bab 2 §2.3](#)), supaya tetap benar dipakai bersama oleh halaman di kedalaman folder yang berbeda-beda.
4. Form Tambah Buku/Anggota kini benar-benar **mengirim data** ke `proses_tambah.php` (bukan lagi form kosong tanpa `action` seperti sejak jobsheet-01).
5. `proses_tambah.php` memvalidasi data di **server**, menyimpannya ke `$_SESSION`, lalu redirect ke halaman daftar.
6. `list.php` merender tabel dari `$_SESSION`, menggantikan pendekatan `fetch` / JSON di jobsheet-06.
7. **Flash message** – pesan sukses/gagal yang muncul sekali setelah redirect.
8. `assets/js/buku.js`, `assets/js/anggota.js`, dan folder `data/` dari jobsheet-06 **dihapus** – tidak dibutuhkan lagi karena rendering sudah pindah ke server.

Daftar Isi

1. [Konsep Dasar PHP](#)
2. [includes/header.php & includes/footer.php](#)
3. [Session & Alur Data](#)
4. [Memproses Form: proses_tambah.php](#)
5. [Menampilkan Data: list.php & Flash Message](#)
6. [CSS: Gaya Flash Message](#)
7. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-07/
├── index.php           # Beranda, kini file PHP
├── includes/
│   ├── header.php     # BARU – bagian atas HTML + navbar, dipakai ulang
│   └── footer.php      # BARU – bagian bawah HTML + footer, dipakai ulang
├── assets/
│   ├── css/style.css  # Ditambah gaya .flash
│   └── js/app.js       # Tidak berubah dari jobsheet-06
└── buku/
```

```

|   └─ list.php           # Render dari $_SESSION, bukan lagi fetch/JSON
|   └─ tambah.php        # Form kini punya method="post" & action
|   └─ proses_tambah.php  # BARU – validasi server + simpan ke $_SESSION
└─ anggota/
    └─ list.php
    └─ tambah.php
    └─ proses_tambah.php  # BARU
└─ docs/wireframe.md     # Identik dengan jobsheet-06
└─ README.md
└─ Dokumentasi/          # Folder dokumentasi ini

```

Catatan penting dari [README.md](#) jobsheet ini yang perlu diingat sejak awal: data yang disimpan di `$_SESSION` **akan** hilang** begitu sesi browser berakhir (menutup browser, atau sesi** kedaluwarsa) – ini jembatan **sementara** menuju penyimpanan sungguhan. Mulai Jobsheet 8, data akan dipindah ke database PostgreSQL supaya benar-benar tersimpan permanen.

1. Konsep Dasar PHP

Ini pengenalan pertamamu ke PHP. Kenali dulu bagaimana PHP berbeda secara **mendasar** dari HTML/CSS/JavaScript yang sudah kamu pelajari.

1.1 Server-Side vs Client-Side: Bedanya Apa?

Ini konsep **paling penting** untuk dipahami sebelum apa pun lainnya:

	Client-side (HTML, CSS, JavaScript)	Server-side (PHP)
Berjalan di mana?	Di browser pengguna (komputer/HP pembaca situs)	Di server (komputer tempat aplikasi web ditempatkan)
Kapan?	Setelah halaman diterima browser	Sebelum halaman dikirim ke browser
Bisa dilihat pengguna?	Ya — lewat "View Source"/DevTools, kode aslinya terlihat	Tidak pernah — pengguna hanya menerima hasil akhirnya (HTML biasa)

	Client-side (HTML, CSS, JavaScript)	Server-side (PHP)
Contoh di SIMPUS-Mini	app.js mengatur menu hamburger (dokumentasi jobsheet-05)	proses_tambah.php memvalidasi & menyimpan data (bab 4)

Bayangkan mengirim surat lewat kantor pos: **server-side** adalah proses di kantor pos (menyortir, memvalidasi alamat, mencatat) yang **tidak terlihat** oleh pengirim maupun penerima surat — mereka hanya melihat hasilnya (surat terkirim atau dikembalikan). **Client-side** adalah hal-hal yang terjadi setelah surat itu sampai di tangan penerima (misalnya penerima membuka amplop, membaca isinya).

1.2 Sintaks Dasar: Tag `<?php ?>`

```
<?php
$page_title = "Beranda";
?>
```

Kode PHP selalu diapit tag pembuka `<?php` dan tag penutup `?>`. Di **luar** tag ini, semua teks (termasuk HTML biasa) diperlakukan apa adanya, persis seperti file `.html` biasa. Inilah kenapa file `.php` bisa berisi **campuran** HTML dan PHP dalam satu file — perhatikan `buku/list.php` ([bab 5](#)) berisi tag `<section>`, `<table>` HTML biasa, **diselingi** potongan kode PHP di sana-sini.

1.3 Variabel PHP: Selalu Diawali `$`

```
$page_title = "Beranda";
$totalBuku = count($_SESSION['buku'] ?? []);
```

- Setiap **variabel** (tempat menyimpan nilai) di PHP **wajib** diawali tanda dolar (`$`) — beda dengan JavaScript yang memakai `let / const` tanpa simbol khusus (ingat dari [dokumentasi jobsheet-05 §4.2](#)). `$page_title` dan `$totalBuku` adalah dua variabel berbeda.
- Setiap baris pernyataan PHP diakhiri **titik koma** (`;`) — sama seperti kebiasaan yang sudah kamu lihat di JavaScript.

1.4 Menampilkan Nilai ke HTML: `echo`

```
<p><?php echo $totalBuku; ?></p>
```

`echo` adalah perintah PHP untuk "mencetak/menampilkan" sebuah nilai ke output HTML yang dikirim ke browser. Baris ini menghasilkan HTML seperti `<p>12</p>` (kalau nilai `$totalBuku` adalah 12) — perhatikan begitu PHP selesai memproses, **tidak ada jejak PHP tersisa** di HTML yang diterima browser; yang terlihat hanyalah angka biasa di dalam tag `<p>`. Inilah wujud nyata dari "server-side" yang dibahas di [§1.1](#) — proses `echo` terjadi di server, hasilnya (angka 12) yang sampai ke browser.

1.5 Superglobal: Variabel Bawaan yang Selalu Tersedia

PHP punya beberapa variabel spesial yang disebut **superglobal** — selalu tersedia di mana pun tanpa perlu dideklarasikan sendiri, namanya selalu huruf besar semua:

Superglobal	Isinya
<code>\$_SESSION</code>	Data yang disimpan antar halaman untuk satu pengunjung yang sama, selama sesi browsernya masih aktif. Dibahas mendalam di bab 3 .
<code>\$_POST</code>	Data yang dikirim lewat form dengan <code>method="post"</code> (ingat dari dokumentasi jobsheet-01 §4.2 , form sebelumnya tidak punya <code>method</code> sama sekali). Dibahas di bab 4 .

1.6 Operator ?? (Null Coalescing)

```
$flash = $_SESSION['flash'] ?? null;
$totalBuku = count($_SESSION['buku'] ?? []);
```

`??` adalah operator "kalau tidak ada, pakai nilai ini" — mirip konsepnya dengan *optional chaining* `?.` di JavaScript yang sudah kamu kenal dari [dokumentasi jobsheet-05 §5.4](#), tapi cara kerjanya sedikit berbeda: `$_SESSION['flash'] ?? null` berarti "ambil `$_SESSION['flash']` **kalau ada**; kalau kunci itu belum pernah diset sama sekali, pakai `null` sebagai gantinya" — mencegah error "undefined array key" yang akan muncul kalau kunci itu diakses langsung padahal belum pernah ada.

1.7 Menjalankan PHP: Butuh Server Sungguhan

Berbeda dari HTML biasa yang bisa dibuka langsung dengan klik dua kali (`file://`), file `.php` **wajib** diproses oleh **PHP interpreter** sebelum bisa dilihat sebagai halaman web — kalau kamu

membuka `index.php` langsung di browser tanpa server, browser hanya akan menampilkan **kode PHP mentahnya sebagai teks**, bukan hasil akhirnya. Sesuai [README.md](#) jobsheet ini, jalankan:

```
php -S localhost:8000
```

Jalankan perintah ini **dari dalam folder** `jobsheet-07/` supaya `http://localhost:8000/` memetakan langsung ke folder ini. Path CSS/JS/menu di jobsheet ini dihitung **relatif otomatis** terhadap folder proyek (bukan digantung ke akar server) — dijelaskan detail di [bab 2 §2.3](#), jadi selain lewat `php -S`, proyek ini juga tetap bisa dibuka lewat Laragon meskipun diakses bersarang di dalam beberapa folder (misalnya `http://dp2026.test/kode-praktikum/jobsheet-07/`). Setelah server berjalan, buka `http://localhost:8000/index.php` — mirip dengan cara menjalankan jobsheet-06 (ingat [dokumentasi jobsheet-06 §7.3](#)), hanya saja kali ini server yang sama juga **memproses** kode PHP-nya, bukan cuma menyajikan file statis apa adanya.

Dengan bekal konsep dasar ini, kamu siap membaca file-file PHP jobsheet ini mulai bab 2.

Lanjut ke: [includes/header.php](#) & [includes/footer.php](#)

2. includes/header.php & includes/footer.php

Ini fitur PHP pertama yang benar-benar menyelesaikan masalah nyata: sejak jobsheet-01, `<header>` dan `<footer>` yang **persis sama** diulang-ulang di **setiap** file HTML ([dokumentasi jobsheet-01 §2.2](#)). Kalau suatu saat menu navigasi perlu ditambah satu tautan baru, kamu harus mengedit **semua** file satu-satu. PHP `include` menyelesaikan masalah ini sekali untuk selamanya.

2.1 includes/header.php — Kode Lengkap

```
<?php
session_start();

$__jobsheetRoot = dirname(__DIR__);
$__scriptDir = dirname($_SERVER['SCRIPT_FILENAME']);
$__rel = ltrim(str_replace('\\', '/', substr($__scriptDir, strlen($__jobsheetRoot))), '/');
$base = $__rel === '' ? '' : str_repeat('../', substr_count($__rel, '/') + 1);
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1">
<title>SIMPUS-Mini<?php echo isset($page_title) ? ' | ' . $page_title : ''; ?></title>
<link rel="stylesheet" href="<?php echo $base; ?>assets/css/style.css">
</head>
<body>
  <header>
    <h1>SIMPUS-Mini</h1>
    <button type="button" id="nav-toggle-btn" class="nav-toggle-label" aria-label="Menu">&#9776;
    <nav>
      <ul>
        <li><a href="<?php echo $base; ?>index.php">Beranda</a></li>
        <li><a href="<?php echo $base; ?>buku/list.php">Daftar Buku</a></li>
        <li><a href="<?php echo $base; ?>buku/tambah.php">Tambah Buku</a></li>
        <li><a href="<?php echo $base; ?>anggota/list.php">Daftar Anggota</a></li>
        <li><a href="<?php echo $base; ?>anggota/tambah.php">Tambah Anggota</a></li>
      </ul>
    </nav>
  </header>

  <main>

```

Perhatikan file ini **tidak ditutup rapi** — tag `<main>` dibuka tapi tidak pernah ditutup, `<body>` dan `<html>` juga tidak ditutup. Ini **disengaja**, dijelaskan di [§2.4](#).

Baris `$base = ...` di paling atas belum dijelaskan — tunggu dulu, dibahas lengkap di [§2.3](#) setelah kamu paham dulu **masalah** yang membuat baris itu perlu ada.

2.2 Memanggil `include`

```

<?php
$page_title = "Beranda";
include __DIR__ . '/includes/header.php';

```

- `include` adalah pernyataan PHP yang **menyisipkan** seluruh isi file lain **tepat di posisi itu** — efeknya seolah-olah seluruh isi `header.php` "ditempel" langsung menggantikan baris `include` ini.
- `__DIR__` adalah konstanta bawaan PHP yang selalu berisi path folder tempat file yang sedang berjalan berada. Menggabungkannya dengan `'/includes/header.php'` menghasilkan path yang **selalu benar**, tidak peduli dari halaman mana `include` ini dipanggil — beda dengan path relatif (`../`) yang harus dihitung manual tergantung kedalaman folder (ingat perhitungan `../` di [dokumentasi jobsheet-01 §1.5](#)). Bandingkan pemanggilannya di `index.php` (`__DIR__ . '/includes/header.php'`) dengan di `buku/list.php` (`__DIR__ .`

'../includes/header.php') — keduanya benar karena `__DIR__` otomatis menyesuaikan diri dengan lokasi file masing-masing.

- `$page_title = "Beranda";` ditulis **sebelum** baris `include` — urutan ini penting! Variabel yang dibuat di `index.php` **tetap bisa diakses**** di dalam `header.php` setelah di-include (PHP menyatukan** keduanya seolah-olah satu file besar), sehingga baris `<?php echo isset($page_title) ? ' | ' . $page_title : ''; ?>` di dalam `header.php` bisa "melihat" nilai `$page_title` yang baru saja diset oleh halaman yang memanggilnya.

2.3 Path Relatif Otomatis di `includes/header.php`

Bandingkan tautan menu di `header.php` sekarang:

```
<li><a href="<?php echo $base; ?>index.php">Beranda</a></li>
<li><a href="<?php echo $base; ?>buku/list.php">Daftar Buku</a></li>
```

dengan versi `jobsheet-05/06` yang memakai path **relatif** tulis-tangan (`../index.html`, `list.html`, `dst.` — ingat konsepnya dari [dokumentasi jobsheet-01 §1.5](#)).

Kenapa path relatif tulis-tangan tidak cukup di sini? Karena `header.php` sekarang **dipakai bersama** oleh halaman-halaman di kedalaman folder yang **berbeda-beda** (`index.php` di root, `buku/list.php` satu folder lebih dalam). Kalau menu ditulis dengan path relatif tetap seperti di `jobsheet-05/06`, satu tautan yang sama (misalnya ke Beranda) akan butuh `index.php` kalau dipanggil dari root, tapi `../index.php` kalau dipanggil dari `buku/list.php` — **tidak mungkin** menulis satu baris HTML yang benar untuk kedua situasi itu sekaligus dalam satu file `header.php` yang sama.

Solusi yang dipakai: hitung prefix-nya otomatis pakai PHP, bukan ditulis manual. Inilah isi 4 baris yang belum dijelaskan di [§2.1](#):

```
$_jobsheetRoot = dirname(__DIR__);
$_scriptDir = dirname($_SERVER['SCRIPT_FILENAME']);
$_rel = ltrim(str_replace('\\', '/', substr($_scriptDir, strlen($_jobsheetRoot))), '/');
$base = $_rel === '' ? '' : str_repeat('../', substr_count($_rel, '/') + 1);
```

- `$_jobsheetRoot = dirname(__DIR__);` — `__DIR__` di dalam `header.php` selalu berisi folder `includes/` (ingat [§2.2](#)). `dirname()` naik satu tingkat dari situ, jadi `$_jobsheetRoot` adalah folder root proyek ini (`jobsheet-07/`) — **selalu benar**, tidak peduli lewat domain apa proyek ini diakses.
- `$_scriptDir = dirname($_SERVER['SCRIPT_FILENAME']);` — `$_SERVER['SCRIPT_FILENAME']` adalah superglobal lain (mirip `$_SESSION` / `$_POST` dari [bab 1 §1.5](#)) berisi path lengkap file PHP yang

pertama kali dijalankan untuk request ini — misalnya `buku/list.php`, **bukan** `header.php` walaupun kode ini sedang berjalan di dalam `header.php` (ingat lagi `include` di §2.2: setelah *di-include*, semuanya jadi "satu file besar", termasuk nilai `$_SERVER['SCRIPT_FILENAME']`-nya).

- `$_rel = ...` — mengambil bagian path **setelah** folder root proyek. Kalau halaman yang dibuka adalah `index.php`, hasilnya string kosong. Kalau halaman yang dibuka `buku/list.php`, hasilnya `"buku"`. `str_replace('\\', '/', ...)` diperlukan karena Windows memakai `\` sebagai pemisah folder, beda dengan `/` yang dipakai di URL.
- `$base = ...` — kalau `$_rel` kosong, `$base` jadi string kosong (tidak perlu naik folder sama sekali). Kalau `$_rel` berisi satu segmen folder (`"buku"`), `$base` jadi satu `"../"`. Rumus ini otomatis menghasilkan `"../../"`, dst. kalau suatu saat ada halaman yang lebih dalam lagi.

Intinya: `$base` menghitung "**berapa level saya perlu naik untuk** sampai ke root proyek ini", **dihitung dari struktur folder di disk** (lewat `__DIR__` dan `SCRIPT_FILENAME`) — bukan dari alamat domain di browser. Perhitungannya dilakukan **sekali** di `header.php`, hasilnya dipakai berulang di setiap `href / src` lewat `<?php echo $base; ?>` — prinsip yang sama dengan alasan `header.php / footer.php` sendiri dibuat: **jangan ulang pekerjaan yang sama di banyak tempat**.

Konsekuensinya: proyek ini **tidak lagi terikat** harus dijalankan dengan folder `jobsheet-07/` sebagai akar server. `php -S localhost:8000` dari dalam folder `jobsheet-07/` tetap valid dan paling sederhana untuk latihan mandiri — tapi proyek ini **juga** tetap benar diakses lewat Laragon meskipun document root Apache-nya bukan folder `jobsheet-07/` itu sendiri (misalnya `http://dp2026.test/kode-praktikum/jobsheet-07/`), karena `$base` menyesuaikan diri secara otomatis, bukan digantungkan ke akar domain seperti pendekatan *root-relative path* (`/index.php`) yang lebih sederhana tapi mengharuskan server persis di folder proyek.

2.4 Bagaimana Dua Potongan Ini Disatukan?

Ingat dari §2.1, `header.php` membuka tag `<main>` tapi tidak menutupnya. Ini karena **konten spesifik**** tiap halaman (misalnya `<section>` "Selamat Datang..." di** Beranda) ditulis **di antara** pemanggilan `include header.php` dan `include footer.php`:

```
<?php
$page_title = "Beranda";
include __DIR__ . '/includes/header.php';
?>

<section>
    <h2>Selamat Datang di Sistem Perpustakaan Mini</h2>
    ...
</section>

<?php include __DIR__ . '/includes/footer.php'; ?>
```

Dan includes/footer.php:

```
</main>

<footer>
    <p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 7</p>
</footer>
<script src="<?php echo $base; ?>assets/js/app.js"></script>
<?php if (!empty($extra_scripts)): foreach ($extra_scripts as $src): ?>
<script src="<?php echo $src; ?>"></script>
<?php endforeach;
endif; ?>
</body>
</html>
```

Perhatikan footer.php ikut memakai `<?php echo $base; ?>` untuk `<script src="...">`-nya — variabel `$base` yang dihitung di header.php (§2.3) tetap tersedia di sini karena header.php dan footer.php di-include dari file yang sama (§2.2), jadi keduanya berbagi variabel yang sama persis seolah-olah satu file besar.

footer.php **menutup** apa yang dibuka header.php: `</main>`, `<footer>`, lalu `</body></html>`. Ketika PHP memproses index.php, urutan penggabungannya:

1. Seluruh isi header.php (buka `<html> ... <main>`).
2. Konten `<section>` khusus Beranda yang ditulis di index.php.
3. Seluruh isi footer.php (`</main> ...tutup </html>`).

Hasil akhirnya: **satu dokumen HTML utuh dan valid** yang dikirim ke browser — persis strukturnya seperti file HTML statis di jobsheet-jobsheet sebelumnya, hanya saja sekarang "dirakit" dari 3 potongan file berbeda di server, bukan ditulis manual berulang di setiap file.

2.5 Pola `$extra_scripts` untuk Script Tambahan per Halaman

```
<?php if (!empty($extra_scripts)): foreach ($extra_scripts as $src): ?>
<script src="<?php echo $src; ?>"></script>
<?php endforeach;
endif; ?>
```

Ini contoh **sintaks alternatif** PHP untuk struktur kontrol — `if (...): ... endif;` dan `foreach (...): ... endforeach;` alih-alih kurung kurawal `{ }` biasa. Sintaks ini **sengaja dipilih** untuk kode PHP yang bercampur dengan HTML (seperti di sini) karena lebih mudah dibaca dibanding kurung kurawal yang bisa membingungkan di antara tag-tag HTML — pola ini akan sering muncul lagi di

[bab 5](#). `!empty($extra_scripts)` memeriksa apakah variabel ini **ada isinya** (variabel ini sendiri belum pernah dipakai secara aktif di jobsheet-07 — disiapkan sebagai "kait" untuk kebutuhan mendatang, mirip semangat menyiapkan struktur lebih awal yang sudah kamu lihat di [dokumentasi jobsheet-04](#) saat merancang sebelum coding).

Lanjut ke: [Session & Alur Data](#)

3. Session & Alur Data

Sebelum membedah `proses_tambah.php` dan `list.php`, pahami dulu **ke mana**** data buku/anggota sebenarnya "disimpan" di jobsheet ini — karena** jawabannya cukup mengejutkan untuk pemula.

3.1 Apa itu Session?

Session adalah cara server "mengingat" seorang pengunjung **antar** halaman**, bahkan antar kunjungan berbeda dalam rentang waktu tertentu.** Protokol HTTP (dasar komunikasi web) sebenarnya bersifat **stateless** (tidak mengingat apa pun) — setiap kali browser meminta sebuah halaman, server memperlakukannya sebagai permintaan yang **benar-benar baru**, tanpa tahu apakah permintaan sebelumnya datang dari orang yang sama. Session menyelesaikan ini: server memberi setiap pengunjung sebuah "tanda pengenalan" unik (biasanya lewat cookie tersembunyi di browser), supaya server bisa mencocokkan "oh, permintaan ini datang dari pengunjung yang sama seperti tadi" dan mengambilkan data yang relevan.

3.2 Mengaktifkan Session: `session_start()`

```
<?php
session_start();
?>
```

Ingat baris ini dari [bab 2 §2.1](#) — berada di **baris paling atas** `header.php`. `session_start()` **wajib** dipanggil di **setiap** halaman yang ingin menggunakan `$_SESSION` (ingat superglobal ini dari [bab 1 §1.5](#)), dan **harus** dipanggil **sebelum** ada output apa pun dikirim ke browser (karena itulah letaknya di baris paling atas file, sebelum `<!DOCTYPE html>`). Karena setiap halaman meng-`include` `header.php` ([bab 2 §2.2](#)), `session_start()` otomatis terpanggil di **semua** halaman tanpa perlu menulisnya berulang-ulang — manfaat lain dari pola `include` yang sudah dibahas di [bab 2](#).

3.3 \$_SESSION sebagai "Keranjang" Data Sementara

Bayangkan `$_SESSION` sebagai sebuah **keranjang** yang disediakan server khusus untuk **satu pengunjung**, dan keranjang itu **tetap ada** selama pengunjung itu berpindah-pindah halaman (selama sesi browsernya aktif). Di jobsheet ini, ada 2 "slot" yang dipakai di keranjang itu:

Kunci	Isinya	Diisi oleh
<code>\$_SESSION['buku']</code>	Array berisi semua buku yang sudah ditambahkan	buku/proses_tambah.php (bab 4)
<code>\$_SESSION['anggota']</code>	Array berisi semua anggota yang sudah ditambahkan	anggota/proses_tambah.php
<code>\$_SESSION['flash']</code>	Pesan sukses/gagal untuk ditampilkan sekali saja	Kedua file <code>proses_tambah.php</code>

3.4 Alur Lengkap: dari Form Sampai Tabel

Mari telusuri **seluruh perjalanan** data dari saat kamu mengisi form Tambah Buku sampai muncul di tabel Daftar Buku:

1. Kamu membuka `buku/tambah.php`, mengisi form, klik "Simpan".
2. Browser mengirim data form (lewat `method="post"`, dibahas di [bab 4 §4.2](#)) ke `buku/proses_tambah.php`.
3. `proses_tambah.php` **memvalidasi** data itu ([bab 4](#)). Kalau valid: data itu ditambahkan ke **array** `$_SESSION['buku']` (data buku yang sudah ada sebelumnya **tidak hilang** — ingat `$_SESSION` bersifat "keranjang", data baru ditambahkan ke situ, bukan menyimpannya).
4. `proses_tambah.php` mengatur `$_SESSION['flash']` berisi pesan sukses, lalu **mengalihkan** (redirect) browser ke `buku/list.php`.
5. `buku/list.php` dibuka (sebagai permintaan halaman **baru**, terpisah dari langkah sebelumnya) — ia membaca `$_SESSION['buku']` (yang sekarang **sudah berisi** buku yang baru saja ditambahkan di langkah 3. dan `$_SESSION['flash']` (pesan sukses dari langkah 4), lalu merender keduanya jadi tabel HTML dan pesan flash (dibahas di [bab 5](#)).

Perhatikan **setiap langkah di atas adalah permintaan HTTP terpisah** ke server (langkah 2, langkah 5 masing-masing satu kali "kunjungan" baru) — dan `$_SESSION` inilah **satu-satunya** cara data (buku yang baru ditambahkan, pesan flash) bisa "menyeberang" dari satu permintaan ke

permintaan berikutnya, karena seperti dibahas di [§3.1](#), HTTP sendiri tidak mengingat apa pun antar permintaan.

3.5 Kenapa Data Ini "Sementara"?

Ingat catatan penting di [README.md](#) jobsheet ini: data di `$_SESSION` akan **hilang** begitu sesi browser berakhir (menutup browser sepenuhnya, membersihkan cookie, atau sesi kedaluwarsa di server setelah periode tidak aktif). Ini **jembatan sementara** — bukan penyimpanan data yang sesungguhnya. Bandingkan dengan `data/buku.json` di jobsheet-06 ([dokumentasi jobsheet-06 §3](#)): file JSON itu **tetap ada** di folder, tidak peduli sesi browser mana pun yang mengaksesnya — tapi juga **tidak bisa ditambah** datanya lewat form (hanya bisa dibaca, tidak ditulis). `$_SESSION` di jobsheet-07 **bisa ditambah** lewat form, tapi **tidak permanen** antar sesi. Kombinasi "bisa ditambah dan permanen" baru akan tercapai mulai Jobsheet 8 dengan database PostgreSQL sungguhan.

Lanjut ke: [Memproses Form: proses_tambah.php](#)

4. Memproses Form: `proses_tambah.php`

Ini file paling penting untuk dipahami di jobsheet ini — tempat data form benar-benar **diproses** untuk pertama kalinya sejak jobsheet-01.

4.1 Form yang Kini Benar-Benar "Hidup"

Ingat dari [dokumentasi jobsheet-01 §4.2](#), tag `<form>` sejak jobsheet-01 **tidak** punya `action / method`, sehingga menekan "Simpan" tidak melakukan apa-apa. Sekarang:

```
<form id="form-tambah" method="post" action="proses_tambah.php">
```

- `method="post"` — menentukan **cara** data dikirim. Data yang dikirim lewat `POST` **tidak terlihat** di address bar browser (beda dengan `method="get"` yang menempelkan data ke URL, cara yang lazim dipakai untuk pencarian). `POST` lebih cocok untuk form yang menambah/mengubah data, seperti menambah buku baru.
- `action="proses_tambah.php"` — menentukan **ke mana** data dikirim ketika form di-submit. Perhatikan ini path **relatif polos** (tanpa prefix `$base` seperti menu navigasi di [bab 2 §2.3](#)) — karena `proses_tambah.php` **selalu** berada tepat di folder yang sama dengan `tambah.php` yang memanggilnya (`buku/tambah.php` → `buku/proses_tambah.php`, `anggota/tambah.php` →

anggota/proses_tambah.php), jadi path relatif sederhana sudah cukup di sini, tidak perlu `__DIR__` atau perhitungan `$base`.

4.2 Menerima Data Form: `$_POST`

```
<?php
session_start();

$judul = trim($_POST['judul'] ?? '');
$pengarang = trim($_POST['pengarang'] ?? '');
$tahun = $_POST['tahun'] ?? '';
$isbn = trim($_POST['isbn'] ?? '');
$stok = $_POST['stok'] ?? '';
$kategori = trim($_POST['kategori'] ?? '');
```

- `$_POST['judul']` — mengambil nilai field bernama `judul` yang dikirim lewat form (ingat atribut `name="judul"` pada `<input>` sejak [dokumentasi jobsheet-01 §4.3](#) — `$_POST` mengambil datanya persis berdasarkan nama atribut `name` itu).
- `?? ''` — operator null coalescing yang sudah dibahas di [bab 1 §1.6](#): kalau field itu entah bagaimana tidak terkirim sama sekali, gunakan string kosong sebagai gantinya, mencegah error.
- `trim(...)` — menghapus spasi kosong di awal/akhir teks (persis konsep yang sama dengan `.trim()` di JavaScript yang sudah kamu pakai di [dokumentasi jobsheet-05 §7.6](#)). Perhatikan `$tahun` dan `$stok` **tidak** di-`trim()` — karena keduanya akan diperiksa sebagai **angka** ([§4.3](#)), bukan teks, jadi spasi di sekitarnya tidak relevan untuk diperiksa dengan cara yang sama seperti field teks.

4.3 Validasi Server-Side

```
$errors = [];
if ($judul === '') {
    $errors[] = "Judul wajib diisi.";
}
if ($pengarang === '') {
    $errors[] = "Pengarang wajib diisi.";
}
if (!is_numeric($tahun) || $tahun < 1900 || $tahun > 2026) {
    $errors[] = "Tahun harus di antara 1900-2026.";
}
if (!is_numeric($stok) || $stok < 0) {
```

```
$errors[] = "Stok tidak boleh negatif.";
}
```

- `$errors = [];` — membuat sebuah **array kosong**, tempat menampung daftar pesan kesalahan yang ditemukan (sintaks `[]` di PHP modern setara dengan `array()`).
- `$errors[] = "...";` — menambahkan satu item baru ke **akhir** array `$errors` (tanda kurung siku kosong `[]` di sisi kiri berarti "tambahkan item baru", bukan mengakses index tertentu).
- `is_numeric($tahun)` — memeriksa apakah nilai `$tahun` **berupa** angka** (atau teks yang bisa dianggap angka, seperti "2005").** Fungsi ini setara konsepnya dengan pengecekan `isNaN()` yang sudah kamu pakai di JavaScript ([dokumentasi jobsheet-05 §7.6](#)), hanya dengan logika terbalik (`is_numeric` mengembalikan `true` kalau memang angka, `isNaN` mengembalikan `true` kalau **bukan** angka).
- Pola pemeriksaan rentang tahun (1900-2026) dan stok non-negatif **persis meniru** aturan yang sama yang sudah ada sejak [dokumentasi jobsheet-01 §4.4](#) (atribut HTML `min/max`) dan divalidasi ulang di JavaScript sejak [dokumentasi jobsheet-05 §7.6](#) — sekarang ditulis untuk **ketiga kalinya**, kali ini di server. Alasan kenapa ini **tidak sia-sia** dijelaskan di [§4.6](#).

4.4 Kalau Ada Error: Simpan Flash & Redirect Kembali

```
if (!empty($errors)) {
    $_SESSION['flash'] = ['type' => 'error', 'pesan' => implode(' ', $errors)];
    header('Location: tambah.php');
    exit;
}
```

- `!empty($errors)` — memeriksa apakah array `$errors` **berisi** sesuatu (ada minimal satu pesan kesalahan yang ditambahkan tadi).
- `$_SESSION['flash'] = ['type' => 'error', 'pesan' => ...];` — menyimpan **array asosiatif** (array dengan kunci bernama, mirip objek di JavaScript) ke `$_SESSION['flash']` — ingat konsep ini dari [bab 3 §3.3](#). Kunci `'type'` (`'error'` atau `'success'`) menentukan **jenis** pesan (dipakai untuk styling, lihat [bab 6](#)), kunci `'pesan'` berisi **teks** pesannya.
- `implode(' ', $errors)` — menggabungkan **semua** item di array `$errors` menjadi **satu string**, dipisah spasi (`' '`). Kalau ada 2 error ("Judul wajib diisi." dan "Tahun harus di antara 1900-2026."), hasilnya jadi satu kalimat gabungan: "Judul wajib diisi. Tahun harus di antara 1900-2026.".
- `header('Location: tambah.php');` — perintah PHP untuk mengirim **HTTP redirect** ke browser: instruksi "tolong buka halaman `tambah.php` sebagai gantinya." Browser akan otomatis pindah ke halaman itu tanpa campur tangan pengguna.

- `exit;` — menghentikan eksekusi skrip PHP **seketika itu juga**. Ini **wajib** ditulis setelah `header('Location: ...')` — tanpa `exit`, PHP akan **tetap melanjutkan** menjalankan baris-baris kode berikutnya (termasuk kode menyimpan data di §4.5) meskipun perintah redirect sudah "dikirim", karena `header()` hanya mengatur instruksi HTTP, bukan benar-benar menghentikan program seperti `return` pada fungsi.

4.5 Kalau Valid: Simpan ke Session & Redirect ke Daftar

```
if (!isset($_SESSION['buku'])) {  
    $_SESSION['buku'] = [];  
}  
  
$_SESSION['buku'][] = [  
    'judul' => $judul,  
    'pengarang' => $pengarang,  
    'tahun' => (int) $tahun,  
    'isbn' => $isbn,  
    'stok' => (int) $stok,  
    'kategori' => $kategori,  
];  
  
$_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku berhasil ditambahkan.'];  
header('Location: list.php');  
exit;
```

- `if (!isset($_SESSION['buku'])) { $_SESSION['buku'] = []; } — memeriksa apakah $_SESSION['buku'] sudah pernah dibuat sebelumnya (isset = "apakah sudah diset"). Kalau ini adalah buku pertama yang ditambahkan sepanjang sesi ini, $_SESSION['buku'] belum ada sama sekali — baris ini membuatnya sebagai array kosong terlebih dulu, supaya baris berikutnya ($_SESSION['buku'][] = ...) bisa langsung menambahkan item ke dalamnya tanpa error.`
- `$_SESSION['buku'][] = [...]` — menambahkan **satu array** asosiatif baru** (representasi satu buku, dengan kunci-kunci yang** **sama persis** dengan struktur `data/buku.json` di jobsheet-06, ingat dari [dokumentasi jobsheet-06 §3.1](#)) — ke **akhir** array `$_SESSION['buku']`, tanpa menghapus buku-buku yang sudah ditambahkan sebelumnya.
- `(int) $tahun` dan `(int) $stok` — ini **type casting**, memaksa nilai untuk diperlakukan sebagai **tipe integer** (angka bulat), bukan lagi teks. Ingat dari [dokumentasi jobsheet-06 §3.4](#) pentingnya membedakan angka dari teks — data yang dikirim lewat `$_POST` **selalu** berupa teks apa adanya (meskipun terlihat seperti angka), jadi `(int)` memastikan nilai `tahun / stok` yang tersimpan di `$_SESSION` benar-benar bertipe angka, bukan teks "2005".

- Redirect ke `list.php` (bukan `tambah.php` seperti kasus error) — membawa pengguna langsung melihat **hasil** dari data yang baru saja ditambahkan.

4.6 Kenapa Validasi Ini yang Benar-Benar Bisa Diandalkan?

Ingat catatan penting dari [dokumentasi jobsheet-05 §7.8](#): validasi HTML (`required`, `min` / `max`) dan validasi JavaScript sama-sama berjalan di **browser pengguna**, dan keduanya **bisa dilewati**** (menonaktifkan JavaScript, atau mengirim data langsung tanpa** lewat form sama sekali). Validasi di `proses_tambah.php` ini **berbeda** secara mendasar**: karena berjalan di **server** ([bab 1 §1.1](#)),** kode ini **selalu** dijalankan untuk **setiap** data yang masuk, apa pun cara pengiriman datanya — pengguna tidak punya cara untuk "mematikan" kode PHP di server, tidak seperti mematikan JavaScript di browser sendiri.

Coba buktikan sendiri sesuai catatan di [README.md](#) jobsheet ini: **nonaktifkan JavaScript** di pengaturan browser, lalu buka `buku/tambah.php` dan submit form kosong. Validasi JavaScript ([dokumentasi jobsheet-05 §7](#)) tidak akan berjalan sama sekali (karena JS dimatikan), tapi form tetap akan gagal tersimpan dan kamu diarahkan kembali ke `tambah.php` dengan pesan error — bukti nyata bahwa validasi server bekerja **independen** dari validasi client.

Lanjut ke: [Menampilkan Data: `list.php` & Flash Message](#)

5. Menampilkan Data: `list.php` & Flash Message

Ini adalah sisi lain dari alur data yang sudah dipetakan di [bab 3 §3.4](#) — bagaimana `$_SESSION['buku']` yang sudah diisi `proses_tambah.php` ([bab 4](#)) akhirnya benar-benar tampil sebagai tabel di layar.

5.1 Kode Lengkap `buku/list.php`

```
<?php
$page_title = "Daftar Buku";
include __DIR__ . '/../includes/header.php';

$flash = $_SESSION['flash'] ?? null;
unset($_SESSION['flash']);
$daftarBuku = $_SESSION['buku'] ?? [];
```

?>

```
<section>
    <h2>Daftar Buku</h2>

    <?php if ($flash): ?>
        <p class="flash flash-?><?php echo $flash['type']; ?>"><?php echo $flash['pesan']; ?><
    <?php endif; ?>

    <div class="search-box">
        <label for="search-input">Cari Judul Buku</label>
        <input type="text" id="search-input" placeholder="Ketik judul buku...">
    </div>

    <div class="table-responsive">
        <table>
            <thead>
                <tr>
                    <th>Judul</th>
                    <th>Pengarang</th>
                    <th>Tahun</th>
                    <th>Stok</th>
                    <th>Aksi</th>
                </tr>
            </thead>
            <tbody>
                <?php if (empty($daftarBuku)): ?>
                    <tr>
                        <td colspan="5">Belum ada data buku. Silakan tambah lewat menu "Tambah Buku"
                    </tr>
                <?php else: ?>
                    <?php foreach ($daftarBuku as $buku): ?>
                        <tr>
                            <td><?php echo $buku['judul']; ?></td>
                            <td><?php echo $buku['pengarang']; ?></td>
                            <td><?php echo $buku['tahun']; ?></td>
                            <td><?php echo $buku['stok']; ?></td>
                            <td>
                                <button type="button">Edit</button>
                                <button type="button" class="btn-hapus">Hapus</button>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
```

```
</section>
<?php include __DIR__ . '/../includes/footer.php'; ?>
```

5.2 Mengambil dan "Menghapus" Flash Message

```
$flash = $_SESSION['flash'] ?? null;
unset($_SESSION['flash']);
```

- Baris pertama mengambil `$_SESSION['flash']` yang mungkin sudah diset oleh `proses_tambah.php` ([bab 4 §4.4-4.5](#)), atau `null` kalau belum ada apa pun.
- `unset($_SESSION['flash']);` — menghapus kunci 'flash' dari `$_SESSION` **seketika setelah dibaca**. Ini kunci dari konsep "flash message" — pesan yang **hanya muncul sekali**. Kalau baris `unset` ini tidak ada, pesan "Buku berhasil ditambahkan." akan **terus** muncul** setiap kali kamu membuka ulang `list.php` di kemudian hari** (karena data itu masih tersimpan di `$_SESSION`), padahal pesan itu seharusnya hanya relevan **tepat setelah** aksi tambah data terjadi. Dengan `unset` segera setelah dibaca, pesan itu otomatis "sekali pakai."

5.3 Menampilkan Flash Message di HTML

```
<?php if ($flash): ?>
    <p class="flash flash-<?php echo $flash['type']; ?>"><?php echo $flash['pesan']; ?></p>
<?php endif; ?>
```

- `<?php if ($flash): ?>` — ingat sintaks alternatif ini dari [bab 2 §2.5](#): kalau `$flash` berisi sesuatu (bukan `null`), paragraf ini ditampilkan; kalau tidak, seluruh blok ini **dilewati sepenuhnya** — tidak ada `<p>` kosong yang muncul di HTML kalau tidak ada pesan flash.
- `class="flash flash-<?php echo $flash['type']; ?>"` — perhatikan **dua** class ditulis sekaligus, dipisah spasi: `flash` (gaya dasar, sama untuk semua flash message) dan `flash-error / flash-success` (gaya spesifik, tergantung nilai `$flash['type']` yang diset di `proses_tambah.php`). Kalau `$flash['type']` adalah 'success', hasil akhirnya jadi `class="flash flash-success"`. Penjelasan CSS untuk kedua class ini ada di [bab 6](#).

5.4 Menampilkan Tabel dari Array Session

```
<?php if (empty($daftarBuku)): ?>
<tr>
    <td colspan="5">Belum ada data buku. Silakan tambah lewat menu "Tambah Buku".</td>
</tr>
```

```

<?php else: ?>
    <?php foreach ($daftarBuku as $buku): ?>
    <tr>
        <td><?php echo $buku['judul']; ?></td>
        <td><?php echo $buku['pengarang']; ?></td>
        <td><?php echo $buku['tahun']; ?></td>
        <td><?php echo $buku['stok']; ?></td>
        <td>
            <button type="button">Edit</button>
            <button type="button" class="btn-hapus">Hapus</button>
        </td>
    </tr>
    <?php endforeach; ?>
<?php endif; ?>

```

- `empty($daftarBuku)` — memeriksa apakah array `$daftarBuku` **kosong** (belum ada satu pun buku ditambahkan sepanjang sesi ini). Kalau kosong, tampilkan **satu baris pesan** ("Belum ada data buku...") memakai `colspan="5"` (ingat atribut ini dari [dokumentasi jobsheet-06 §4.7](#) — konsep yang persis sama, hanya sekarang ditangani di PHP, bukan JavaScript).
- `foreach ($daftarBuku as $buku): ... endforeach;` — mengulang **setiap** elemen di array `$daftarBuku`, dengan `$buku` mewakili satu buku pada tiap putaran. Bandingkan langsung dengan `daftarBuku.forEach(function (buku) { ... })` yang sudah kamu pakai di JavaScript ([dokumentasi jobsheet-06 §4.6](#)) — **konsep yang identik** (mengulang setiap item array), hanya sintaks bahasanya berbeda.
- `<?php echo $buku['judul']; ?>` — mengambil nilai dari kunci `'judul'` pada array asosiatif `$buku`, sama persis strukturnya dengan yang disimpan `proses_tambah.php` ([bab 4 §4.5](#)).

5.5 Perbandingan Besar: Rendering di Server vs di Browser

Ini pergeseran konseptual paling penting di jobsheet ini:

	Jobsheet-06 (Fetch/JSON)	Jobsheet-07 (PHP)
Di mana tabel "dirakit"?	Di browser , oleh JavaScript (<code>buku.js</code>)	Di server , oleh PHP (<code>list.php</code>)
Sumber data	File <code>data/buku.json</code> , diambil lewat <code>fetch()</code>	<code>\$_SESSION['buku']</code> , dibaca langsung di server
HTML yang diterima	<code><tbody></code> kosong dulu, lalu diisi belakangan oleh JS	<code><tbody></code> sudah terisi penuh sejak awal — hasil akhir jadi, siap tampil

Jobsheet-06 (Fetch/JSON)

Jobsheet-07 (PHP)

browser		
Butuh loading indicator?	Ya (ingat dokumentasi jobsheet-06 §4.3)	Tidak — data sudah "jadi" sebelum halaman dikirim, tidak ada jeda yang terlihat pengguna

Perhatikan `buku/list.php` **tidak lagi punya** elemen `#loading-indicator` maupun memuat `buku.js` — karena tidak ada lagi proses "menunggu" yang terlihat oleh pengguna: begitu `list.php` selesai diproses server, HTML yang dikirim **sudah lengkap** berisi seluruh baris tabel, siap ditampilkan seketika oleh browser.

Lanjut ke: [CSS: Gaya Flash Message](#)

6. CSS: Gaya Flash Message

Satu-satunya perubahan `style.css` di jobsheet ini — mendukung tampilan flash message yang sudah dibahas di [bab 5 §5.3](#).

6.1 Kode CSS

```
/* ===== Flash Message ===== */
.flash {
  padding: 0.75rem 1rem;
  border-radius: 6px;
  margin-bottom: 1rem;
  font-weight: 500;
}

.flash-success {
  background-color: #d4edda;
  color: #155724;
}

.flash-error {
  background-color: #f8d7da;
  color: #721c24;
}
```

6.2 Gaya Dasar (`.flash`)

`.flash` berisi gaya yang **selalu sama** untuk kedua jenis pesan: padding nyaman, sudut membulat (`border-radius` , ingat konsepnya dari [dokumentasi jobsheet-02 §5.3](#)), jarak di bawahnya, dan teks yang sedikit tebal (`font-weight: 500` , konsisten dengan bobot yang dipakai label form sejak [dokumentasi jobsheet-02 §8.3](#)).

6.3 Gaya Spesifik per Jenis (`.flash-success` , `.flash-error`)

Ingat dari [bab 5 §5.3](#), elemen flash message selalu punya **dua** class sekaligus: `flash` dan salah satu dari `flash-success` / `flash-error` . Kedua class tambahan ini memberi **warna berbeda** tergantung jenis pesannya:

Class	Warna Latar	Warna Teks	Kesan
<code>.flash-success</code>	#d4edda (hijau sangat muda)	#155724 (hijau tua)	Positif — data berhasil disimpan
<code>.flash-error</code>	#f8d7da (merah muda pucat)	#721c24 (merah tua/marun)	Peringatan — ada yang perlu diperbaiki

Pola warna hijau=sukses, merah=gagal ini konsisten dengan konvensi warna yang sudah kamu pakai sejak awal — ingat warna merah pada tombol Hapus dan pesan error validasi ([dokumentasi jobsheet-05 §3.2](#)) memakai kombinasi merah yang senada (`#d9534f`) untuk maksud yang sama: menandakan sesuatu yang butuh perhatian pengguna.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

7. Rangkuman & Latihan Lanjutan

7.1 Rangkuman Keseluruhan Jobsheet 7

Bagian	Konsep yang Dipelajari
Konsep Dasar PHP	Server-side vs client-side, tag <code><?php ?></code> , variabel <code>\$</code> , <code>echo</code> , superglobal, <code>??</code>
Includes Header/Footer	<code>include</code> , <code>__DIR__</code> , path relatif otomatis (<code>\$base</code>), sintaks alternatif <code>if/foreach: endif/endforeach</code>
Session & Alur Data	<code>session_start()</code> , <code>\$_SESSION</code> sebagai "keranjang" data antar halaman, alur form → proses → tabel
Proses Tambah & Validasi Server	<code>\$_POST</code> , <code>is_numeric</code> , <code>header('Location: ...') + exit</code> , type casting (<code>int</code>)
List.php & Flash Message	<code>unset()</code> untuk pesan sekali-pakai, <code>foreach</code> PHP, rendering di server vs browser
CSS Flash Message	Konvensi warna sukses/gagal

7.2 Konsep Inti yang Perlu Diingat

1. **Server-side berjalan sebelum halaman sampai ke browser** — kode PHP tidak pernah terlihat pengguna, hanya hasilnya ([bab 1 §1.1](#)).
2. `include` **menghapus duplikasi kode** antar halaman — sekali ubah `header.php`, semua halaman yang meng-include-nya ikut berubah ([bab 2](#)).
3. `$_SESSION` **menjembatani data antar permintaan HTTP yang terpisah** — tapi sifatnya sementara, hilang saat sesi browser berakhir ([bab 3](#)).
4. **Validasi server-side tidak bisa dilewati pengguna**, berbeda dari validasi HTML/JavaScript yang keduanya berjalan di browser dan bisa dinonaktifkan ([bab 4 §4.6](#)).
5. **Redirect setelah POST** (`header('Location: ...') + exit`) adalah pola umum untuk mencegah data ter-submit ulang kalau pengguna me-refresh halaman hasil, dan untuk langsung mengarahkan ke halaman yang relevan (form kalau error, daftar kalau sukses) ([bab 4 §4.4-4.5](#)).
6. **Flash message adalah pola "sekali tampil"** — disimpan lalu segera dihapus (`unset`) setelah dibaca, supaya tidak muncul berulang di kunjungan berikutnya ([bab 5 §5.2](#)).

7.3 Cara Mencoba Sendiri

1. Jalankan server-nya (ingat [bab 1 §1.7](#) dan [bab 2 §2.3](#) — path-nya sudah relatif otomatis, jadi tidak wajib dari folder tertentu):

```
php -S localhost:8000
```

Buka `http://localhost:8000/index.php` (atau lewat Laragon, lihat [README.md](#) untuk opsi vhost-nya).

2. Klik "Tambah Buku", isi form dengan data valid, klik "Simpan" — amati kamu diarahkan ke `list.php` dengan pesan hijau "Buku berhasil ditambahkan." di atas tabel, dan buku barumu muncul di baris terakhir.
3. Refresh halaman `list.php` — amati pesan hijau tadi **sudah tidak** muncul lagi** (bukti `unset()` dari** [bab 5 §5.2](#) bekerja), tapi data bukunya **tetap ada** (karena `$_SESSION['buku']` tidak ikut dihapus, hanya `$_SESSION['flash']`).
4. Coba submit form kosong — amati pesan **merah** muncul di `tambah.php`, dan tidak ada data baru yang tersimpan.
5. Praktikkan uji dari [README.md](#) jobsheet ini: nonaktifkan JavaScript di pengaturan browser, submit form kosong lagi — buktikan validasi server tetap bekerja meski JavaScript mati total (ingat [bab 4 §4.6](#)).
6. Tutup browser sepenuhnya (bukan cuma tab-nya), buka lagi — amati data buku yang tadi kamu tambahkan **sudah hilang** (bukti sifat sementara `$_SESSION` dari [bab 3 §3.5](#)).

7.4 Ide Latihan Tambahan (Opsional)

1. **Tambah validasi ISBN** di `buku/proses_tambah.php` — misalnya memastikan ISBN yang diisi (kalau tidak kosong) hanya berisi angka dan tanda hubung, memakai fungsi PHP `preg_match()`.
2. **Tambah flash message** di `anggota/proses_tambah.php` untuk kasus yang belum ditangani — bandingkan dengan versi `buku/proses_tambah.php` yang sudah divalidasi lebih lengkap (rentang tahun, stok non-negatif) — field apa lagi di form anggota yang mungkin perlu aturan validasi tambahan?
3. **Buat halaman** `debug_session.php` sementara (untuk latihan, hapus setelah selesai) yang menampilkan isi `$_SESSION` mentah lewat `<pre><?php print_r($_SESSION); ?></pre>` — cara yang berguna untuk "mengintip" langsung apa yang sebenarnya tersimpan di server saat belajar.
4. **Tambah tombol "Reset Data"** yang memanggil `session_destroy()` untuk mengosongkan seluruh `$_SESSION` secara manual, tanpa perlu menutup browser — cari tahu sendiri lewat dokumentasi PHP resmi bagaimana fungsi ini bekerja.

Kalau ada bagian yang masih membingungkan, coba baca ulang [bab 3](#) sambil mempraktikkan langkah 2-3 di [§7.3](#) — melihat sendiri data "bertahan" antar halaman lalu "hilang" setelah browser ditutup adalah cara paling efektif untuk benar-benar memahami apa itu session.